

LeetCode Notes: Range Sum Query

© Authored by: Arpit Raj, LNMIIT Jaipur | August 14, 2026

Concept: Square Root Decomposition

This problem is structurally identical to "**13E holes of codeforces**", making it an ideal candidate for **Square Root Decomposition**. By dividing the array into blocks of size $\sqrt{n}$, we can optimize both updates and range sums.

Point Update Logic

When updating an element at a specific index, we must simultaneously update its corresponding block sum:

1. `blocksum[blockidx] -= arr[index];` — *purani hatao* (remove old)
2. `blocksum[blockidx] += val;` — *nayi lagao* (add new)

Range Sum Logic (The Most Important Part)

To calculate the sum across a range seamlessly, view the target sequence divided into three distinct sections:

.....1.....B B.....B B.....B B.....r....B

1. Left Alignment

"pehle left ko kisi block ke start me laane tak ka sum"

.....1.....B

2. Complete Blocks

"phir whole blocks ka sum"

B.....B B.....B

3. Right Remainder

"phir bache hue ka sum"

B.....r....B

Implementation Code

```
class NumArray {
public:

    //this question is just like
    //13E holes of codeforces
    //so im using square root decomposition

    vector<int> arr;
    vector<int> blocksum;
    int B;

    NumArray(vector<int>& nums) {

        arr=nums;
        int n=arr.size();

        B=sqrt(n)+1;
        blocksum.resize(B,0);

        for(int i=0;i<n;i++){
            blocksum[i/B]+=nums[i];
        }

    }

    void update(int index, int val) {
        int blockidx=index/B;
        blocksum[blockidx]-=arr[index]; //purani hatao
        blocksum[blockidx]+=val; //nayi lagao
        arr[index]=val;
    }

    int sumRange(int left, int right) {

        //most important part
        //pehle left ko kisi block ke start me laane tak ka sum
        //phir whole blocks ka sum
        //phir bache hue ka sum

        //basically see this like
        // ....1....B   B.....B B.....B   B....r...B

        //so 1 to B
        //2 complete blocks
        //B to r

        int ans=0;
        while(left<=right && left%B!=0){
```

```

        ans+=arr[left];
        left++;
    }

    while(left/B < right/B){
        ans+=blocksum[left/B];
        left+=B;
    }

    while(left<=right){
        ans+=arr[left];
        left++;
    }

    return ans;
}
};

/**
 * Your NumArray object will be instantiated and called as such:
 * NumArray* obj = new NumArray(nums);
 * obj->update(index,val);
 * int param_2 = obj->sumRange(left,right);
 */

```